Deterministic Pre-Context Compression for Multi-Agent Language Model Pipelines via Three-Pass Hilbert Sparse Hypercubes and 64-Byte Fixed-Point Cayley–Dickson Zero-Divisor Algebra

Christopher "Topher" Duguay (0009-0006-8693-6839)
University of Manitoba, Winnipeg, MB, Canada
umdugua2@myumanitoba.ca | https://doi.org/10.5281/zenodo.23873219

November 2, 2026

Abstract

Multi-agent large language model (LLM) pipelines incur linear $\mathcal{O}(L)$ input-token billing and quadratic $\mathcal{O}(L^2)$ self-attention and key-value (KV) cache overhead when streaming raw reference corpora and JSON Lines (`JSONL`) execution trajectories into neural context windows, compounded by invoking secondary neural judge models for structural admission filtering and re-executing uncached tool-call subtrees. We introduce the **`BSA-R` Three-Pass Hilbert Sparse Hypercube Compiler**, a zero-dependency CPU compilation and relational slicing engine paired with `64-byte` `Q8.8` fixed-point `16D` Sedenion and `32D` Pathion Cayley–Dickson registers. Each `PathionQ88` state vector packs 32 signed 16-bit fixed-point lanes (2^{-8} scale) into a single 64-byte CPU L1 cache line alongside a 128-byte bit-packed exclusive-OR (`XOR`) basis sign lookup table (`S32`), enabling: (i) a *Pre-Context Zero-Divisor Admission Gate* that annihilates redundant boilerplate channels to literal zero ($G_{\text{adm}} \odot_{8.8} v_{\text{dup}} = 0 \in \mathbb{Z}^{32}$) with bit-exact distributive signal preservation and zero floating-point drift; (ii) *Non-Associative `32D` Pathion AST Memoization* exploiting broken associativity and broken alternativity at dimension $2^k \geq 16$ to detect re-parenthesized and depth-shifted abstract syntax trees (ASTs) missed by commutative bag-of-vectors caches; and (iii) three-pass N-dimensional Hilbert space-filling curve compressed sparse row/column (`CSR/CSC`) hypercube compilation queried through immutable copy-on-modify Base `R 4.6.1` functional pipes (`|>`). Evaluated with exact `tiktoken:o200k_base` tokenization across four production workspace chores, `BSA-R` reduces total suite input volume by 84.60% (6.49×), from 2,467,118 to 380,027 tokens, achieving a 99.87% (782.0×) reduction across corpus cross-referencing, trajectory telemetry, and pre-context admission chores (1,585,118 → 2,027 tokens) and executing zero-divisor admission checks in 35.12 μs on CPU at 0 LLM token cost. Longitudinal re-evaluation on an expanded multi-agent trajectory (+40.1% byte growth) increases suite-wide compression to 84.77% (2,494,872 → 380,083 tokens; 30.14 μs gate latency), cross-validated bit-for-bit in Base `R 4.6.1`.

1 Introduction and Related Work

Modern multi-agent software engineering, scientific auditing, and document concordance harnesses routinely treat the Transformer context window [22] as a general-purpose relational execution engine. In a typical workflow, an orchestrator serializes megabyte-scale primary corpora or hundreds of kilobytes of multi-agent `JSONL` interaction logs directly into prompt text to perform multi-attribute

filtering, cross-table joins, or telemetry aggregation. Simultaneously, control-plane harnesses frequently invoke auxiliary "LLM-as-a-judge" calls on every reasoning step to filter prompt boilerplate or enforce structural guardrails, and issue 5×–10× stochastic temperature retries whenever a tool-calling agent enters a repetitive execution basin.

This architectural pattern creates severe computational and epistemic bottlenecks across both the data plane and the control plane:

1. **Prefill Complexity and KV-Cache Footprint:** Standard causal self-attention over a sequence of length L requires $\mathcal{O}(L^2 d)$ arithmetic operations during prefill and $\mathcal{O}(L)$ high-bandwidth GPU memory for KV-cache storage [22]. Hardware-aware attention kernels such as `FlashAttention` [3] eliminate intermediate $\mathcal{O}(L^2)$ HBM materializations via SRAM tiling, and virtualized block allocators such as `PagedAttention` [16] eliminate KV-cache memory fragmentation. However, both operate *downstream* of tokenization: when 99% of an injected 250,000-token prompt consists of rows irrelevant to a structured query, exact GPU attention still computes quadratic interactions over every unpruned token and bills the caller for the entire $\mathcal{O}(L)$ input payload.

2. **Pre-Compression Inference Overhead and Epistemic Fragility in Neural Pruning:** Perplexity-driven token pruning methods such as `LLMLingua` [13] and `LongLLMLingua` [14] run a smaller auxiliary causal language model over the raw prompt to estimate token-level self-information prior to forwarding compressed text to a frontier model. While effective for natural-language prose summarization, neural prompt pruning still incurs $\mathcal{O}(L)$ neural forward passes before compression, lacks cryptographic provenance receipts, and introduces a non-zero probability of deleting exact line identifiers, numerical weights, or schema delimiters required for deterministic auditing.

3. **Associative Collapse in Dense Retrieval and Embedding Caches:** Retrieval-augmented generation (RAG) [17] bounds prompt length by retrieving top-k dense embedding chunks via inner-product similarity. Yet dense vector similarity cannot execute exact relational group-by roll-ups, anti-echo cluster deduplication, or existential middle-term projections [2, 9]. Worse, when mean-pooled or summed vector embeddings are used as semantic memoization keys for agent tool-call trees, vector addition is both commutative ($A + B = B + A$) and associative ($(A + B) + C = A + (B + C)$), rendering such caches blind to argument transpositions and parenthetical execution-order spoofs.

To eliminate redundant token transmission before any neural API or GPU prefill is invoked, we draw on three mature domains outside neural sequence modeling: sparse tensor compilation, space-filling multidimensional indexing, and deterministic fixed-point systems programming. In high-performance scientific computing, the Tensor Algebra Compiler (`TACO`) [15] and sparse matrix libraries in `R` [12, 1] store only nonzero support coordinates in compressed sparse row and column (`CSR/CSC`) formats, while N-dimensional Hilbert space-filling curves [10, 19] map multidimensional integer coordinates onto one-dimensional contiguous arrays with strong spatial locality. In parallel, multipass 16-bit `W65C816S` macro assemblers—culminating in `Asar`—resolve sparse memory-bank allocations in three deterministic passes (Pass 0 symbol discovery → Pass 1 layout sizing → Pass 2 zero-drift convergence verification) while representing physical quantities in 16-bit signed `Q8.8` fixed-point integers (2^{-8} resolution) to prevent IEEE-754 floating-point rounding drift [8].

Contributions. We synthesize these principles in the **`BSA-R` Three-Pass Hilbert Sparse Hypercube Compiler** (`bsar`), making four technical contributions:

1. **Single-Cache-Line `Q8.8` Cayley–Dickson Zero-Divisor Kernel (Section 2):** We formalize `PathionQ88`, a 64-byte ($32 \times$ `int16`) fixed-point representation of `16D` Sedenions and `32D` Pathions [6, 21, 20, 4, 5] paired with a 128-byte bit-packed `XOR` sign lookup table (`S32`), occupying 192 bytes (3 CPU L1 cache lines, 64× smaller than a 4,096-byte dense `FP32` 32×32 routing matrix). We prove that our *Pre-Context Zero-Divisor Admission Gate* annihilates paired boilerplate blades to 64 literal `0x00` bytes while preserving orthogonal task signal bit-for-bit, and supplies 7 octahedral Box-Kite zero-divisor escape operators replacing blind temperature retries.

2. **Non-Associative and Non-Alternative `32D` Pathion AST Memoization (Section 2.3):** Because Cayley–Dickson algebras of dimension $2^k \geq 16$ break commutativity, associativity, and alternativity [21], recursive left-to-right `PathionQ88` tree folding yields a 64-byte structural memoization key that distinguishes both parenthetical regroupings $((A, B), C) \neq (A, (B, C))$ and repeated-leaf depth shifts $((B, B), C) \neq (B, (B, C))$ with 100.0% empirical accuracy across 240 tool-call ASTs.

3. **Three-Pass Hilbert `CSR/CSC` Hypercube Compiler and Base `R` Functional Pipe Algebra (Section 3):** Using Lua-inspired dynamic ragged records (`LuaDynamicRecord`) [11], `BSA-R` compiles sparse multi-axis corpora and `JSONL` logs in three passes with a strict Pass-2 cardinality lock ($\Delta_{\text{pass}} = 0$) and canonical `SHA-256` provenance seal, exposing $\mathcal{O}(1)$-indexed relational verbs (`filter_eq`, `inner_join`, Carrollian existential axis elimination `eliminate_axis`, and anti-echo `rollup_slice`) interoperable with Base `R 4.6.1` pipes (`|>`) [12, 23].

4. **Reproducible `o200k_base` Empirical Audit (Section 4):** Across four real workspace chores tokenized with `tiktoken:o200k_base`, `BSA-R` eliminates 84.60%–84.77% of suite-wide input tokens (99.87% / 782× on data, telemetry, and admission chores) with microsecond-to-millisecond CPU latency and independent Base `R 4.6.1` verification.

2 Mathematical Preliminaries: `Q8.8` `16D`/`32D` Zero-Divisor Gates and Non-Associative AST Hashing

2.1 The Cayley–Dickson Doubling Tower and `64-Byte` `PathionQ88` Registers

Let $\mathbb{A}_0 = \mathbb{R}$, and for each integer $k \geq 1$ define the 2^k-dimensional real Cayley–Dickson algebra $\mathbb{A}_k = \mathbb{A}_{k-1} \oplus \mathbb{A}_{k-1}$ via recursive involution (conjugation) and bilinear multiplication [6, 21]:

$$(a, b)^* := (a^*, -b), \qquad (a, b)(c, d) := \big(ac - d^*b,\ da + bc^*\big), \qquad a, b, c, d \in \mathbb{A}_{k-1}, \tag{1}$$

with $x^* = x$ on $\mathbb{A}_0$. Each doubling sacrifices an algebraic symmetry: $\mathbb{A}_1 = \mathbb{C}$ loses total ordering; $\mathbb{A}_2 = \mathbb{H}$ (quaternions) loses commutativity; $\mathbb{A}_3 = \mathbb{O}$ (octonions) loses associativity while remaining alternative ($x(xy) = (xx)y$ and $(yx)x = y(xx)$); and for $k \geq 4$—specifically the `16D` Sedenions $\mathbb{S} = \mathbb{A}_4$ and `32D` Pathions $\mathbb{P} = \mathbb{A}_5$—the algebra loses both alternativity and the division-ring property, acquiring structured manifolds of nontrivial zero divisors [21, 20, 4, 5].

IEEE-754 binary32/binary64 floating-point arithmetic is non-associative under addition and sensitive to FMA contraction and SIMD reduction order across CPU microarchitectures [8], which can perturb exact zero-divisor phase cancellation into small nonzero residuals ($\sim 10^{-16}$). To guarantee cross-platform bit-exactness across Python, `R`, WebAssembly, and embedded pipelines while fitting a complete `32D` hypercomplex vector inside a single 64-byte CPU L1 cache line, we quantize $\mathbb{A}_5$ over signed 16-bit `Q8.8` fixed-point integers.

Definition 2.1 (`64-Byte PathionQ88` Register)**.** Let $Q_{\min} = -32768$, $Q_{\max} = 32767$, and unit scale $Q_1 = 256$ ($2^{-8} = 0.00390625$ step resolution, dynamic range $[-128.0000, +127.9961]$). A `PathionQ88` element is an immutable 32-tuple

$$x = (x_0, x_1, \ldots, x_{31}) \in \mathcal{Q}_{8.8}^{32} := [Q_{\min}, Q_{\max}]_{\mathbb{Z}}^{32}, \tag{2}$$

serialized in little-endian two's-complement format (`struct.pack("<32h", *x)`) as exactly $32 \times 2 = 64$ bytes (1 standard L1 cache line). Real scalars $\alpha \in \mathbb{R}$ lift into `Q8.8` via $\mathrm{q88}(\alpha) := \max\big(Q_{\min}, \min(Q_{\max}, \lfloor \alpha Q_1 + 0.5 \rfloor)\big)$ (using round-to-nearest), with canonical basis elements $E_i^{(s)} := s\, e_i \in \mathcal{Q}_{8.8}^{32}$ ($s = Q_1$ by default). Notice that any `16D` Sedenion embeds isomorphically into the lower 16 lanes $(x_0, \ldots, x_{15}, 0, \ldots, 0) \in \mathcal{Q}_{8.8}^{32}$.

Proposition 2.2 (Bitwise `XOR` Basis Product and 128-Byte Sign Table `S32`)**.** *Let $\{e_0, e_1, \ldots, e_{31}\}$ denote the canonical basis of $\mathbb{A}_5$ generated by (1). For all $0 \le i, j < 32$,*

$$e_i e_j = S_{i,j}\, e_{i \oplus j}, \qquad S_{i,j} \in \{-1, +1\}, \tag{3}$$

where $\oplus$ denotes bitwise exclusive-OR on 5-bit indices. Encoding $S_{i,j}$ as single bits $b_{32i+j} = \mathbb{I}[S_{i,j} = -1]$ packs the entire $32 \times 32 = 1{,}024$-entry multiplication table into 1,024 bits $= 128$ bytes (2 CPU L1 cache lines). Defining the sign-symmetric fixed-point shift $\mathcal{T}_{256} : \mathbb{Z} \to [Q_{\min}, Q_{\max}]_{\mathbb{Z}}$ by

$$\mathcal{T}_{256}(a) := \begin{cases} 0, & a = 0, \\ \min\big(Q_{\max},\ \lfloor (a + 128)/256 \rfloor\big), & a > 0, \\ \max\big(Q_{\min},\ -\lfloor (-a + 128)/256 \rfloor\big), & a < 0, \end{cases} \tag{4}$$

the `Q8.8` Pathion product $x \odot_{8.8} y \in \mathcal{Q}_{8.8}^{32}$ has k-th lane ($0 \le k < 32$):

$$(x \odot_{8.8} y)_k = \mathcal{T}_{256}\left(\sum_{i=0}^{31} S_{i,i\oplus k}\, x_i\, y_{i\oplus k} \right) = \mathcal{T}_{256}\left(\sum_{(i,j):\, i\oplus j = k} S_{i,j}\, x_i\, y_j \right). \tag{5}$$

Proof. Equation (3) follows by induction on the doubling step k in (1): splitting 5-bit indices into most-significant half-bit and 4-bit offset maps each block product $(ac, -d^*b, da, bc^*)$ onto $(i \oplus j)$ with unit sign ± 1. Because $j \mapsto i \oplus j$ is a bijection on $\{0, \ldots, 31\}$ for every fixed i, substituting $j = i \oplus k$ collects all pairs contributing to e_k into the exact 64-bit integer accumulator $\mathrm{acc}_k = \sum_{i=0}^{31} S_{i,i\oplus k} x_i y_{i\oplus k}$ prior to a single symmetric rounding shift $\mathcal{T}_{256}(\mathrm{acc}_k)$. □

Notably, whereas a dense 32×32 single-precision (`FP32`) linear router requires $32 \times 32 \times 4 = 4{,}096$ bytes, a `PathionQ88` state register requires 64 bytes (64.0× state compression) and the combined register-plus-`S32` kernel occupies $64 + 128 = 192$ bytes (3 L1 cache lines). Moreover, active-lane sparse iteration ($\{i : x_i \neq 0\} \times \{j : y_j \neq 0\}$ in `cd_mult_q88`) reduces sparse blade multiplications to 4 integer multiply-accumulates.

2.2 Box-Kite Zero-Divisor Geometry and the Pre-Context Admission Gate

Following Moreno [20] and de Marrais [4, 5], a primitive *assessor* in the `16D` Sedenion subalgebra $\mathbb{S} \subset \mathbb{P}$ is a 2-plane spanned by (e_i, e_j) where $i \in \{1, \ldots, 7\}$ is an octonion imaginary index and $j \in \{8, \ldots, 15\}$ is a sedenion imaginary index excluding the pure octonion-doubling partner $j = i \oplus 8$ (since e_i and $e_{i\oplus 8}$ span an associative quaternion subalgebra free of zero divisors). There are therefore $7 \times (8 - 2) = 42$ primitive assessors (as $j = 8 = e_8$ also pairs only trivially). Two unit

diagonal blades $u = e_i + e_j$ and $v = e_k + \sigma e_l$ ($\sigma \in \{-1, +1\}$, $i \neq k$) annihilate ($u \cdot v = 0$) if and only if their cross-terms land on identical `XOR` target lanes with opposite signs:

$$i \oplus k = j \oplus l, \qquad S_{i,k} + \sigma S_{j,l} = 0, \qquad \sigma S_{i,l} + S_{j,k} = 0. \tag{6}$$

Exhaustive constructive evaluation of (6) over `S32` (`enumerate_sedenion_box_kites` in `bsar/cd_q88.py` and `validate_bsar_cd.R`) yields 42 **active assessors** linked by 168 **directed zero-divisor annihilation edges** (matching the order of the Klein quartic automorphism group $|\mathrm{PSL}(2,7)| = 168$), which decompose into 7 **connected octahedral components** (*Box-Kites*) of 6 assessor vertices each. Extending to `32D` Pathions ($\mathbb{P} = \mathbb{A}_5$) preserves all 7 embedded Sedenion Box-Kites while introducing higher-dimensional "Sand Mandala" zero divisors [5], such as $(e_1 + e_{18})(e_4 - e_{23}) = 0$.

Theorem 2.3 (Exact `Q8.8` Bloat Annihilation and Distributive Signal Invariance)**.** *Let (i, j) and (k, l, σ) be any canonical Box-Kite zero-divisor partner pair satisfying (6) with $1 \leq i, k \leq 7$, $9 \leq j, l \leq 15$, and $j \neq i \oplus 8$, $l \neq k \oplus 8$ (for instance, Box-Kite #1: $(i, j) = (1, 10)$ and $(k, l, \sigma) = (4, 15, -1)$). Define the Pre-Context Zero-Divisor Admission Gate (`PorchZeroDivisorGate`) operator $G_{\mathrm{adm}} := Q_1(e_i + e_j) \in \mathcal{Q}_{8.8}^{32}$. For any scalar task signal amplitude $q_s \in [-16383, 16383]_{\mathbb{Z}}$ placed on the real identity lane $s = q_s e_0$ and any redundant/duplicate boilerplate amplitude $q_b \in [Q_{\min}, Q_{\max}]_{\mathbb{Z}}$ placed on the partner blade $v_{\mathrm{dup}} = q_b(e_k + \sigma e_l)$:*

1. ***Exact 64-Byte Annihilation:*** $G_{\mathrm{adm}} \odot_{8.8} v_{\mathrm{dup}} = (0, 0, \ldots, 0) \in \mathbb{Z}^{32}$.
2. ***Bit-Exact Distributive Signal Invariance:*** *Provided $|q_s| \leq Q_{\max}$,*

$$G_{\mathrm{adm}} \odot_{8.8} (s + v_{\mathrm{dup}}) = G_{\mathrm{adm}} \odot_{8.8} s = q_s(e_i + e_j) \in \mathcal{Q}_{8.8}^{32}, \tag{7}$$

so the filtered 64-byte output is bit-for-bit invariant to q_b across its entire dynamic range.

Proof. In (5), multiplying $G_{\mathrm{adm}} = Q_1(e_i + e_j)$ by $s + v_{\mathrm{dup}} = q_s e_0 + q_b(e_k + \sigma e_l)$ produces unshifted integer accumulators supported on at most four target indices:

- Target lanes $i \oplus 0 = i$ and $j \oplus 0 = j$ receive $Q_1 S_{i,0} q_s = 256 q_s$ and $Q_1 S_{j,0} q_s = 256 q_s$ because $S_{m,0} = +1$ for all m.
- Target lane $t_1 := i \oplus k = j \oplus l$ receives $Q_1 q_b(S_{i,k} + \sigma S_{j,l}) = 0$ by (6).
- Target lane $t_2 := i \oplus l = j \oplus k$ receives $Q_1 q_b(\sigma S_{i,l} + S_{j,k}) = 0$ by (6).

Because $S_{i,k} + \sigma S_{j,l} = 0$ and $\sigma S_{i,l} + S_{j,k} = 0$ hold over $\mathbb{Z}$, the v_{dup} contribution cancels identically in the unshifted integer accumulators prior to applying $\mathcal{T}_{256}$ (and moreover $\{t_1, t_2\} \cap \{i, j\} = \emptyset$ since $k, l \neq 0$ and $l \notin \{0, i \oplus j\}$). Setting $q_s = 0$ gives $\mathrm{acc}_m = 0$ for all $m \in \{0, \ldots, 31\}$, and since $\mathcal{T}_{256}(0) = 0$, $G_{\mathrm{adm}} \odot_{8.8} v_{\mathrm{dup}} = 0 \in \mathbb{Z}^{32}$. For general $|q_s| \leq Q_{\max}$, $\mathrm{acc}_i = \mathrm{acc}_j = 256 q_s$ and $\mathrm{acc}_m = 0$ otherwise, whence $\mathcal{T}_{256}(256 q_s) = q_s$, establishing (7). □

To guarantee strict operational reversibility, `filter_with_clutch` caches the raw pre-gate 64-byte frame alongside the post-gate output (`clutch_rollback_64b`), allowing zero-loss audit inspection while eliminating 2,500-token neural guardrail invocations per step. Furthermore, iterating over all 7 Box-Kites yields 7 mutually distinct deterministic zero-divisor escape operators ($z_m \odot_{8.8} w_m = 0$, $m = 1, \ldots, 7$), replacing 5× blind stochastic LLM temperature retries with a single orthogonal basin hop when an agent loop stalls.

2.3 Non-Associative 32D Pathion AST Memoization

Consider an agent orchestrator executing composite tool-call DAGs or expression ASTs such as $T_{\text{left}} = ((A, B), C)$ versus $T_{\text{right}} = (A, (B, C))$, or repeated-tool depth variants $U_{\text{left}} = ((B, B), C)$ versus $U_{\text{right}} = (B, (B, C))$. An additive bag-of-embeddings baseline $\Sigma_{\text{bag}}(T) = \sum_{\tau \in \text{leaves}(T)} \psi(\tau)$ satisfies both commutativity and associativity, causing $\Sigma_{\text{bag}}(T_{\text{left}}) \equiv \Sigma_{\text{bag}}(T_{\text{right}})$ and $\Sigma_{\text{bag}}(U_{\text{left}}) \equiv \Sigma_{\text{bag}}(U_{\text{right}})$ (false-positive cache collisions that return stale results from a differently ordered execution plan). Even non-commutative quaternion ($\mathbb{H}$) folding fails on T_{left} vs. T_{right} because $\mathbb{H}$ is associative, and octonion ($\mathbb{O}$) folding fails on U_{left} vs. U_{right} because $\mathbb{O}$ is alternative: $(b \cdot b) \cdot c = b \cdot (b \cdot c)$ for all $b, c \in \mathbb{O}$ [21].

In `BSA-R`, each leaf symbol $\tau \in \Sigma^*$ is lifted into a full-support `32D PathionQ88` vector (`leaf_token_to_pathion_` using the 32-byte `SHA-256` digest $(d_0, \ldots, d_{31}) = \text{SHA256}(\tau)$ [18]:

$$\tilde{c}_r(\tau) = \begin{cases} 2(d_r - 128), & d_r \neq 128, \\ +64, & d_r = 128 \text{ and } r \text{ even}, \\ -64, & d_r = 128 \text{ and } r \text{ odd}, \end{cases} \qquad \psi(\tau) = \mathcal{N}_\infty\big(\tilde{c}(\tau)\big), \tag{8}$$

where the integer L_∞ peak normalizer $\mathcal{N}_\infty(x)_r := \text{round}\big(Q_1 x_r / \max_m |x_m|\big)$ (for $x \neq 0$) keeps every intermediate fold bounded at peak magnitude $Q_1 = 256$, preventing both 16-bit integer overflow and multi-depth underflow to zero. An internal AST tuple $T = (T_1, T_2, \ldots, T_m)$ ($m \geq 2$) is folded left-to-right via `fold_ast_q88`:

$$\Phi(\tau) := \psi(\tau), \qquad \Phi\big((T_1, \ldots, T_m)\big) := \mathcal{N}_\infty\Big(\cdots \mathcal{N}_\infty\big(\Phi(T_1) \odot_{8.8} \Phi(T_2)\big) \cdots \odot_{8.8} \Phi(T_m)\Big). \tag{9}$$

Because $\psi(\tau)$ populates all 32 Pathion coordinates with nonzero signed entries ($|\tilde{c}_r| \geq 2$), non-commutativity ($A \odot_{8.8} B \neq B \odot_{8.8} A$), non-associativity ($[A, B, C]_{8.8} := (A \odot_{8.8} B) \odot_{8.8} C - A \odot_{8.8} (B \odot_{8.8} C) \neq 0$), and broken alternativity ($(B \odot_{8.8} B) \odot_{8.8} C \neq B \odot_{8.8} (B \odot_{8.8} C)$) engage simultaneously at every interior node, yielding a compact 64-byte structural cache key.

3 Three-Pass Hilbert CSR/CSC Hypercube Compilation and R Pipe Algebra

3.1 Three-Pass Assembler Architecture (Pass 0 → Pass 1 → Pass 2)

Let $\mathcal{A} = (A_1, \ldots, A_D)$ be an ordered D-tuple of discrete relational or epistemic axes. Naive multidimensional array materialization allocates $\prod_{d=1}^{D} |V_d|$ cells, which becomes prohibitive on sparse multi-agent logs and cross-referenced text corpora. Generalizing the three-pass symbol-resolution pipeline of `Asar` and `TACO`'s sparse level formats [15], `BsaRHypercubeCompiler` executes three deterministic CPU passes:

1. **Pass 0 (Dynamic Ragged Support Discovery via `LuaDynamicRecord`):** Modeled on Lua's minimal associative table representation [11], each input entity r instantiates a `LuaDynamicRecord` holding arbitrary key-value metadata `attrs` and only its attested sparse coordinate cells $c \in \mathcal{E}_r$. Each cell carries axis values $(c[A_1], \ldots, c[A_D])$, quantized weight $w_{8.8} = \text{q88}(w)$, anti-echo provenance identifier `cluster_id`, and a four-valued epistemic state $\text{state}(c) \in \{0, 1, ?, *\}$ (`CONTRADICTED_ZERO`, `ATTESTED_ONE`, `OPEN_UNKNOWN`, `PROJECTED_STAR`). Iterating over sorted record identifiers computes each record's 64-byte `PathionQ88` attribute-AST fingerprint and collects active axis vocabularies V_d with sorted ordinal bijections $\omega_d : V_d \to \{0, \ldots, |V_d| - 1\}$.

Each cell is assigned an N-dimensional Hilbert key [10, 19] $h(c) = H_D\big(\omega_1(c[A_1]), \ldots, \omega_D(c[A_D])\big)$ by folding pairwise 2D Hilbert indices $H_2 : \{0, \ldots, 2^{16} - 1\}^2 \to \mathbb{Z}_{\geq 0}$ on a 65,536 × 65,536 power-of-two grid.

2. **Pass 1 (Hilbert Locality Ordering and Multi-Axis `CSR/CSC` Inverted Indices):** The sparse cell array $\mathcal{E}^{(0)}$ is sorted by the total lexicographic key

$$\kappa(c) := \big(h(c),\ (c[A_1], \ldots, c[A_D]),\ c[\texttt{record_id}]\big), \tag{10}$$

yielding the contiguous Hilbert-ordered slab $\mathcal{E}^{(1)} = (c_0, c_1, \ldots, c_{M-1})$. A single linear sweep over $\mathcal{E}^{(1)}$ populates per-axis inverted posting lists $I_d(v) = \{idx \in \{0, \ldots, M-1\} : c_{idx}[A_d] = v\}$ for every $d \in \{1, \ldots, D\}$ and $v \in V_d$, equivalent to compressed sparse row/column (`CSR/CSC`) offset-index tables across all D hypercube modes simultaneously.

3. **Pass 2 (Zero-Drift Convergence Lock and Cryptographic `SHA-256` Seal):** Pass 2 enforces the multipass cardinality invariant

$$\Delta_{\text{pass}} := \big||\mathcal{E}^{(1)}| - |\mathcal{E}^{(0)}|\big| = 0, \tag{11}$$

aborting compilation if $\Delta_{\text{pass}} \neq 0$, records the exact count of avoided empty Cartesian cells $\prod_{d=1}^{D} \max(1, |V_d|) - M$, and seals the canonically sorted JSON hypercube under a deterministic digest `rats_sha256`.

3.2 Copy-on-Modify Relational Pipe Algebra and Carrollian Projection

Queries against a compiled slab execute through an immutable copy-on-modify pipeline `RPipe` (mirroring Base `R`'s native `|>` operator and `dplyr` relational verbs [12, 23]):

$$\mathcal{S}\ \texttt{|>}\ \texttt{filter_eq}\ \texttt{|>}\ \texttt{inner_join}\ \texttt{|>}\ \texttt{eliminate_axis}\ \texttt{|>}\ \texttt{rollup_slice}\ \texttt{|>}\ \texttt{to_compact_prompt_table}. \tag{12}$$

Four algebraic properties distinguish (12) from ad-hoc text chunking:

- **$\mathcal{O}(1)$-Indexed Multi-Axis Selection (`filter_eq`):** Given equality constraints $\{A_d = v_d\}_{d \in Q}$ on a compiled slab, `filter_eq` intersects the precomputed Pass-1 posting sets $\bigcap_{d \in Q} I_d(v_d)$ directly without scanning unrelated corpus blocks, materializing only matching cells in sorted Hilbert order.

- **Keyed Relational Enrichment (`inner_join`):** Joins sparse cells against external dictionary tables on a shared key without mutating upstream frames.

- **Carrollian Existential Middle-Term Projection (`eliminate_axis`, $\exists m$):** Inspired by Lewis Carroll's biliteral and triliteral diagram elimination of syllogistic middle terms, eliminating axis A_m over retained key axes $K \subset \mathcal{A} \setminus \{A_m\}$ groups cells by $\pi_K(c)$, marks $c'[A_m] \leftarrow$ `"*"` (`PROJECTED_STAR`), aggregates the eliminated coordinate witnesses into a lossless audit list `expanded_`$A_m = [v_1, v_2, \ldots]$ (preserving full operational reversibility), and computes the tropical max-weight envelope $w'_{8.8}(g) = \max_{c:\pi_K(c)=g} w_{8.8}(c)$.

- **Anti-Echo Integer `Q8.8` OLAP Roll-Up (`rollup_slice`):** Standard OLAP `GROUP BY SUM` [9] over agent logs or cross-referenced corpora suffers from "citation echo" when an interval spread or multi-step retry emits multiple cells sharing a single underlying provenance cluster. Within each group tuple g over axes $G \subseteq \mathcal{A}$, `rollup_slice` filters to attested cells (state$(c) =$

"1"), deduplicates within each `cluster_id` γ via $w^*(g,\gamma) = \max_{c \in g,\, \mathrm{cid}(c)=\gamma} w_{8.8}(c)$, sums $W_{8.8}(g) = \sum_\gamma w^*(g,\gamma)$ in exact integers (eliminating floating-point summation-order non-determinism), and emits compact Markdown/TSV slices via `to_compact_prompt_table`.

Listing 1 illustrates the corresponding zero-dependency Base `R 4.6.1` implementation of the `Q8.8 32D` Pathion product and expense-auditing pipe verb (`r/validate_bsar_cd.R` and `r/cayley_dickson_verbs`

Listing 1: Base `R 4.6.1 Q8.8 32D` Pathion `XOR` product and functional pipe audit (`r/validate_bsar_cd.R`, `r/cayley_dickson_verbs.R`).

```
trunc_q88 <- function(acc) {
  ifelse(acc == 0L, 0L,
    ifelse(acc > 0L, bitwShiftR(acc + 128L, 8L), -bitwShiftR((-acc) + 128L, 8L)))
}

cd_mult_q88 <- function(x, y) {
  vapply(0:31, function(k) {
    j <- bitwXor(0:31, k) + 1L
    acc <- sum(as.integer(x) * as.integer(y[j]) * S32[cbind(1:32, j)])
    as.integer(trunc_q88(acc))
  }, integer(1))
}

# Verify native 32D Pathion zero divisor (e_1 + e_18) * (e_4 - e_23) == 0 in Q8.8:
u32 <- e_basis(1L) + e_basis(18L)
v32 <- e_basis(4L) - e_basis(23L)
stopifnot(all(cd_mult_q88(u32, v32) == 0L))
```

4 Empirical Evaluation on `o200k_base` Workloads

4.1 Benchmark Suite and Experimental Protocol

We evaluate `BSA-R` using the reproducible benchmark harness `bsar/chore_bench.py` on a standard Windows x86-64 workstation running Python 3.14 and Base `R 4.6.1`, measuring exact Byte-Pair Encoding (BPE) token counts via OpenAI's production `tiktoken:o200k_base` tokenizer across four representative multi-agent workloads:

1. **Chore 1 (`CHORE_1_CORPUS_5Q` — 1.13 MB Primary Corpus 5-Query Cross-Reference):** Analytical lookup across a 1,134,681-byte (20,061-line, 253,531-`o200k_base`-token) English doctrinal reference corpus (`rc-en.txt`). A naive full-context agent harness streams the raw corpus across 5 analytical topic queries ($5 \times 253{,}531 = 1{,}267{,}655$ input tokens). `BSA-R` segments the text into 250-line blocks indexed over $D = 3$ axes (`pillar`, `topic_tag`, `chunk_band`), compiles a three-pass Hilbert slab once, and executes `filter_eq |> eliminate_axis("chunk_band") |> to_compact_prompt_table(max_rows=8)` per query.

2. **Chore 2 (`CHORE_2_TRANSCRIPT_AUDIT` — Multi-Agent `transcript.jsonl` Telemetry and Compaction):** Observability and token-spend rollup over a real multi-agent execution log (`transcript.jsonl`) indexed over $D = 4$ axes (`source`, `step_type`, `tool_name`, `status`) with step output kilotokens stored in `Q8.8`. We evaluate *two* exported longitudinal snapshots of this live workspace trajectory: **Snapshot A** (`dist/cerebral_valley_benchmark.json`, 223,758 bytes, 67,463 raw tokens) and **Snapshot B** (`docs/compiler-hud/cerebral_valley_benchmark.js`, 313,410 bytes, 95,217 raw tokens after +40.1% trajectory growth).

3. **Chore 3 (`CHORE_3_PORCH_ZD_FIREWALL` — 100-Step Pre-Context Zero-Divisor Admission Gate):** Evaluates 100 agent control-plane steps in which a standard harness invokes a 2,500-token LLM guardrail/filter prompt per step (250,000 tokens total) versus `PorchZeroDivisorGate` (1,000 timed `Q8.8` iterations on mixed signal $\alpha = 3.25$ and boilerplate $\beta = 9.75$).

4. **Chore 4 (`CHORE_4_AST_CACHE_AND_BOXKITE` — 240-AST `64-Byte` Pathion Memoization and 7-Box-Kite Escape):** Evaluates 80 cold canonical 3-leaf tool-call ASTs $((a, b_i), c)$, 80 exact repeat invocations, 40 parenthetically regrouped adversarial ASTs $(a, (b_i, c))$, and 40 repeated-leaf depth-shifted ASTs $((b_i, b_i), c)$ vs. $(b_i, (b_i, c))$, plus 10 stuck episodes (4,200 tokens/call; $160 + 10 \times 5 = 210$ naive LLM calls vs. $80 + 10 \times 1 = 90$ `BSA-R` calls).

4.2 Token Compression, CPU Latency, and Longitudinal Trajectory Scaling

Table 1 reports the complete token and latency measurements for baseline Snapshot A (`dist/cerebral_valley_ben` report `SHA-256: 33a75058dc0bd469a09ef46101f2e2c58732ff27e24817a147858c56d9066234`). Across all four chores, `BSA-R` reduces input token consumption by 84.60% **(6.49×)**, saving 2,087,091 `o200k_base` tokens per suite run. On Chores 1–3—where structured selection, trajectory aggregation, and pre-context admission replace raw context stuffing and LLM judge calls entirely—`BSA-R` compresses 1,585,118 naive tokens down to 2,027 tokens, a 99.872% **(782.0×) reduction**.

Table 1: Empirical `tiktoken:o200k_base` token compression and CPU execution latency on baseline Snapshot A (`dist/cerebral_valley_benchmark.json`, `SHA-256: 33a75058...`).

Workspace Chore	Naive Tokens	`BSA-R` Tokens	Tokens Saved	Reduction (Factor)	CPU Late
1. Corpus 5-Query Slice (`1.13 MB`)	1,267,655	1,847	1,265,808	99.854% (686.3×)	112.71 ms
2. Trajectory Audit (`223.8 KB JSONL`)	67,463	180	67,283	99.733% (374.8×)	62.60 ms t
3. 100-Step `Q8.8` Admission Gate	250,000	0	250,000	100.000% (∞)	35.12 μs/g
Subtotal (Chores 1–3: Data/Log/Gate)	1,585,118	2,027	1,583,091	99.872% *(782.0×)*	< 0.18 *s c*
4. 240-AST Cache & 7-Box-Kite Escape	882,000	378,000	504,000	57.143% (2.33×)	319.52 ms
Total 4-Chore Suite (Snapshot A)	**2,467,118**	**380,027**	**2,087,091**	**84.60% (6.49×)**	**Sub-seco**

Table 2 compares Snapshot A (`dist/`) against expanded Snapshot B (`docs/compiler-hud/cerebral_valley_b` report `SHA-256: 6b05a0cd396758930f5dad5f76b7b22d452806feb1730895cf2d75eb73b37a25`). As the live multi-agent `transcript.jsonl` file grew by +40.1% in size (223,758 → 313,410 bytes; 67,463 → 95,217 naive tokens), the compiled `BSA-R rollup_slice` output expanded by only +3 distinct tool-signature rows (9 → 12 rows; 180 → 236 tokens, +56 tokens), improving Chore 2 compression from 99.733% (374.8×) to 99.752% (403.5×) and increasing overall 4-chore compression from 84.60% to 84.77% **(6.56×, 2,114,789 tokens saved)** with a 30.14 μs mean zero-divisor gate latency.

4.3 Adversarial AST Spoof Detection, Asymptotic Complexity, and `R 4.6.1` Audit

Table 3 contrasts our 64-byte `PathionQ88` AST folder against the associative/commutative bag-of-vectors baseline across Chore 4's adversarial tree benchmarks, while Table 4 summarizes asymptotic time, memory, pre-compression neural cost, and epistemic guarantees across competing context-management paradigms.

Table 2: Longitudinal multi-agent trajectory scaling between Snapshot A (`dist/`) and Snapshot B (`docs/compiler-hud/`), verified identically by `r/validate_bsar_cd.R` under Base `R 4.6.1`.

Metric / Verification Invariant	Snapshot A (`dist/`)	Snapshot B (`docs/compiler-hud/`)
Report `SHA-256` Prefix	`33a75058dc0bd469...`	`6b05a0cd39675893...`
Chore 2 `transcript.jsonl` Size	223,758 bytes	313,410 bytes (+40.07%)
Chore 2 Slab `rats_sha256` Prefix	`faada4e5a3016783...`	`5dafc7a6749332a7...`
Chore 2 Naive → `BSA-R` Tokens	67,463 → 180 (−99.733%)	95,217 → 236 (−99.752%)
Chore 1 Compile / Avg. Slice Latency	112.71 ms / 1,027.10 μs	100.34 ms / 974.44 μs
Chore 3 Zero-Divisor Gate Latency	35.12 μs (0 tokens)	30.14 μs (0 tokens)
Chore 4 240-AST Fold Latency	319.52 ms (1.33 ms/AST)	285.37 ms (1.19 ms/AST)
Chores 1–3 Data/Log/Gate Tokens	1,585,118 → 2,027 (782.0×)	1,612,872 → 2,083 (774.3×)
Suite Total Naive → `BSA-R` Tokens	**2,467,118 → 380,027** (**−84.60%**)	**2,494,872 → 380,083** (**−84.77%**)

Table 3: Structural AST memoization and adversarial spoof detection on Chore 4 (240 trees).

Evaluation Criterion	Associative Bag-of-Vectors	`32D PathionQ88`
Exact Repeat Cache Hit Rate (80 identical ASTs)	Unsafe (collides on permutations)	80/80 (100.0% sa
Non-Associative Regrouping $((A,B),C)$ vs. $(A,(B,C))$ (40 pairs)	0.0% detected (0/40)	**100.0**% detected
Broken Alternativity $((B,B),C)$ vs. $(B,(B,C))$ (40 pairs)	0.0% detected (0/40)	**100.0**% detected
Stuck-Loop Basin Escape (7 Sedenion Box-Kites)	5× blind temperature retries	**7/7** exact 64 B zer

Table 4: Asymptotic complexity and provenance comparison for context length L, sparse support $M \ll L$, sliced output $K \ll M$, and query count Q.

Paradigm	Pre-API Cost	LLM Prefill	Cache / Index Space	Provenance & Exactness
Naive Full-Context [22]	$\mathcal{O}(1)$	$\mathcal{O}(QL^2d)$	$\mathcal{O}(QL)$ KV-cache	Exact raw text; prohibitiv
Attention IO / Paged KV [3, 16]	$\mathcal{O}(1)$	$\mathcal{O}(QL^2d)$ FLOPs	$\mathcal{O}(L)$ HBM (tiled)	Exact attention; 0% API
Dense Vector RAG [17]	$\mathcal{O}(L)$ GPU embed	$\mathcal{O}(QK^2d)$	$\mathcal{O}(Ld_{\text{emb}})$ FP32	Approximate top-k; no O
Neural Pruning (`LLMLingua`) [13]	$\mathcal{O}(QL)$ aux. LLM	$\mathcal{O}(QK^2d)$	Auxiliary LLM weights	Probabilistic token drop;
`BSA-R` (Ours)	$\mathcal{O}(M \log M)$ CPU once	$\mathcal{O}(QK^2d)$	$\mathcal{O}(MD) + 192$ B L1	**Bit-exact `Q8.8` + `SHA-2`**

Executing the standalone zero-dependency cross-validator `Rscript r/validate_bsar_cd.R` under `R 4.6.1` independently reconstructs `S32` from (1), verifies $(e_1 + e_{18}) \odot_{8.8} (e_4 - e_{23}) = 0 \in \mathbb{Z}^{32}$ across all 32 lanes, confirms `Assessors = 42` and `Edges = 168`, and reproduces both `Suite Reduction = 84.60% (2467118 -> 380027 tokens)` on `dist/cerebral_valley_metrics.csv` and `Suite Reduction = 84.77% (2494872 -> 380083 tokens)` on `docs/compiler-hud/cerebral_valley_metrics`

5 Limitations, Broader Impact, Reproducibility, and Acknowledgements

Limitations and Scope Boundaries. Three explicit scope conditions govern `BSA-R`:

1. *Complementary Role to Downstream Neural Reasoning:* `BSA-R` deterministically executes multi-axis selection, relational joins, existential middle-term elimination, anti-echo aggregation, pre-context boilerplate cancellation, and exact AST caching on CPU, but open-ended abstractive prose synthesis and semantic entailment over the resulting compact slices (180–1,847 tokens) still require downstream LLM inference.

2. *Signed `Q8.8` Dynamic Range:* Because each 16-bit `Q8.8` lane spans $[-128.0000, +127.9961]$ at 2^{-8} precision, raw unscaled counters exceeding 32,767 clamp if inserted directly into a single lane; applications must either pre-scale wide-range quantities into natural units (such as kilotokens `out_tok / 1000.0` in Chore 2), accumulate roll-ups in widenable host integers (`sum_q88` in `rollup_slice`), or apply L_∞ peak normalization (`normalize_linf_q88` in deep AST folds).

3. *Structural Axis Extraction:* Compiling a `LuaDynamicRecord` hypercube presupposes deterministic structural anchors (JSON keys, document section headers, or AST nodes); wholly unsegmented raw OCR streams require upstream lexical chunking prior to Pass 0 registration.

Broader Impact. Shifting relational filtering, trajectory compaction, structural guardrail admission, and tool-DAG caching from GPU-hosted neural inference to a 192-byte L1-cache-resident CPU kernel reduces cloud API token consumption by 6.5× overall and 782× on data and telemetry workloads. Beyond direct financial savings for academic laboratories and small engineering teams, eliminating millions of redundant prefill tokens proportionally reduces datacenter HBM bandwidth contention and energy expenditure while replacing opaque neural prompt mutations with cryptographically auditable `SHA-256` hypercube receipts.

Reproducibility Statement. The complete `BSA-R` Python package (`bsar/cd_q88.py`, `bsar/hypercube.py`, `bsar/chore_bench.py`), Base `R 4.6.1` verification scripts (`r/cayley_dickson_verbs.R`, `r/validate_bsar_cd.R`) unit test suites, and exported `JSON`/`CSV` benchmark artifacts require zero external runtime dependencies (with optional `tiktoken` support for exact `o200k_base` verification), are released under the GNU Affero General Public License v3.0 (`AGPL-3.0-or-later`), and are permanently archived on CERN Zenodo at `doi:10.5281/zenodo.23873219` [7].

Acknowledgements. The three-pass deterministic compilation and `Q8.8` fixed-point architecture of `BSA-R` builds directly upon the 1984–2026 `W65C816S` multipass assembler and hypercomplex algebra lineage. The author gratefully acknowledges the foundational engineering contributions of Bill Mensch and The Western Design Center (`WDC 65816 Macro Assembler`, 1984); Glen Bredon (`Merlin 8/16`, 1986) and Mike Westerfield / The Byte Works (`ORCA/M`, 1986); Intelligent Systems

and Sony NEWS (`IS-SFX`, 1989–1990); Norman Yen (`SNES Assembler`, `TRASM`, and `x816`, 1993–1997); Near / byuu (`xkas`, 2004–2010); the `Trashkas` syntax-drift translator; and Alcaro together with RPGHacker, randomdude999, KungFuFurby, Lui37, p4plus2, Thomas, and the SMWCentral `Asar` maintainers (2011–present), whose three-pass zero-label-drift assembler invariant inspired Pass 0–Pass 2 of `BsaRHypercubeCompiler`. We likewise acknowledge Robert P. C. de Marrais for cataloging the `16D` Sedenion and `32D` Pathion Box-Kite zero-divisor structures, and Ross Ihaka, Robert Gentleman, Hadley Wickham, the R Core Team, and the R Foundation for the copy-on-modify functional data-frame paradigm.

References

[1] D. Bates, M. Maechler, and M. Jagan, *`Matrix`: Sparse and Dense Matrix Classes and Methods*, CRAN: Contributed Packages, The R Foundation, 2000, `doi:10.32614/CRAN.package.Matrix`.

[2] E. F. Codd, A relational model of data for large shared data banks, *Commun. ACM*, **13**(6):377–387, 1970, `doi:10.1145/362384.362685`.

[3] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, FlashAttention: Fast and memory-efficient exact attention with IO-awareness, in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 16344–16359, 2022, `arXiv:2205.14135`.

[4] R. P. C. de Marrais, The 42 assessors and the Box-Kites they fly: Diagonal axis-pair systems of zero-divisors in the Sedenions' 16 dimensions, `arXiv:math/0011260`, 2000.

[5] R. P. C. de Marrais, Flying higher than a Box-Kite: Kite-chain middens, Sand Mandalas, and zero-divisor patterns in 2^n-ions beyond the Sedenions, `arXiv:math/0207003`, 2002.

[6] L. E. Dickson, On quaternions and their generalization and the history of the eight square theorem, *Ann. of Math. (2)*, **20**(3):155–171, 1919, `doi:10.2307/1967865`.

[7] C. Duguay, *`BSA-R` × Eleven-Entry Bookkeeping: Three-Pass Sparse Hypercubes, Constructive `Lean 4` Moving Viscous Ceiling Proofs, and Supercritical 3D Navier–Stokes Interval Certificates* (Version 1.0.0), CERN Zenodo, 2026, `doi:10.5281/zenodo.23873219`.

[8] D. Goldberg, What every computer scientist should know about floating-point arithmetic, *ACM Comput. Surv.*, **23**(1):5–48, 1991, `doi:10.1145/103162.103163`.

[9] J. Gray, S. Chaudhuri, A. Bosworth, A. Layman, D. Reichart, M. Venkatrao, F. Pellow, and H. Pirahesh, Data cube: A relational aggregation operator generalizing group-by, cross-tab, and sub-totals, *Data Min. Knowl. Discov.*, **1**(1):29–53, 1997, `doi:10.1023/A:1009726021843`.

[10] D. Hilbert, Ueber die stetige Abbildung einer Linie auf ein Flächenstück, *Math. Ann.*, **38**(3):459–460, 1891, `doi:10.1007/BF01199431`.

[11] R. Ierusalimschy, L. H. de Figueiredo, and W. Celes Filho, Lua—An extensible extension language, *Softw. Pract. Exper.*, **26**(6):635–652, 1996, `doi:10.1002/(SICI)1097-024X(199606)26:6<635::AID-SPE26>3.0.CO;2-P`.

[12] R. Ihaka and R. Gentleman, R: A language for data analysis and graphics, *J. Comput. Graph. Stat.*, **5**(3):299–314, 1996, `doi:10.1080/10618600.1996.10474713`.

[13] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, LLMLingua: Compressing prompts for accelerated inference of large language models, in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 13358–13376, Association for Computational Linguistics, 2023, doi:10.18653/v1/2023.emnlp-main.825.

[14] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression, in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1658–1677, Association for Computational Linguistics, 2024, doi:10.18653/v1/2024.acl-long.91.

[15] F. Kjolstad, S. Kamil, S. Chou, D. Lugato, and S. Amarasinghe, The tensor algebra compiler, *Proc. ACM Program. Lang.*, **1**(OOPSLA):77:1–77:29, 2017, doi:10.1145/3133901.

[16] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, Efficient memory management for large language model serving with PagedAttention, in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, pp. 611–626, ACM, 2023, doi:10.1145/3600006.3613165.

[17] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020, arXiv:2005.11401.

[18] R. C. Merkle, A digital signature based on a conventional encryption function, in *Advances in Cryptology — CRYPTO '87*, Lecture Notes in Computer Science, vol. 293, pp. 369–378, Springer, 1988, doi:10.1007/3-540-48184-2_32.

[19] B. Moon, H. V. Jagadish, C. Faloutsos, and J. H. Saltz, Analysis of the clustering properties of the Hilbert space-filling curve, *IEEE Trans. Knowl. Data Eng.*, **13**(1):124–141, 2001, doi:10.1109/69.908985.

[20] R. G. Moreno, The zero divisors of the Cayley–Dickson algebras over the real numbers, *Bol. Soc. Mat. Mexicana (3)*, **4**(1):13–28, 1998, arXiv:q-alg/9710013.

[21] R. D. Schafer, On the algebras formed by the Cayley–Dickson process, *Amer. J. Math.*, **76**(2):435–446, 1954, doi:10.2307/2372583.

[22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, Attention is all you need, in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017, arXiv:1706.03762.

[23] H. Wickham *et al.*, Welcome to the Tidyverse, *J. Open Source Softw.*, **4**(43):1686, 2019, doi:10.21105/joss.01686.